To run the program, I have a README file in the basic folder for instructions!

Milestone 1:

Task 1:

2h4w.json, located inside the hw1 folder, describes the topology. Essentially, the setup will allow the packet to pass through switch 2 but not switch 3. But the setting can be changed with s1-runtime.json and s4-runtime.json. So that the packet can be sent through switch3.

Task 2:

- At S1:
  The ingress pipeline first applies the ecmp_group table, which matches on the destination IP address. If there is a match, the set_ecmp_select action computes a hash (using fields such as source IP, destination IP, protocol, and optionally TCP source port) to select one of multiple next hops. The selected value is used to index into the ecmp_nhop table, which sets the next-hop MAC address and output port for load balancing.
- At S2–S4:
  The switches use the ipv4_lpm table, which matches on the destination IP address and forwards packets to the correct port and MAC address based on static entries. This provides destination-based forwarding.
- Key P4 code snippets:

```
table ecmp_group {
    key = { hdr.ipv4.dstAddr : lpm; }
    actions = { set_ecmp_select; drop; }
    size = 1024;
    default_action = drop();
}

action set_ecmp_select(bit<32> ecmp_base, bit<32> ecmp_count) {
    bit<32> hv;
           hash(hv,   HashAlgorithm.crc32,   0w32,   {   hdr.ipv4.srcAddr,
hdr.ipv4.dstAddr, hdr.ipv4.protocol }, ecmp_count);
    meta.ecmp_select = ecmp_base + hv;
}

table ecmp_nhop {
    key = { meta.ecmp_select : exact; }
    actions = { set_nhop; drop; }
```

```
    size = 64;
    default_action = drop();
}

table ipv4_lpm {
    key = { hdr.ipv4.dstAddr : lpm; }
    actions = { ipv4_forward; drop; }
    size = 1024;
    default_action = drop();
}
```

- Pipeline logic:
    - If the packet matches in ecmp_group, ECMP load balancing is performed at S1.
    - Otherwise, or at S2–S4, the packet is forwarded based on the destination IP using ipv4_lpm.

This design ensures that only S1 performs ECMP load balancing, while S2–S4 use destination-based forwarding.

By running "`for i in $(seq 1 200); do python send.py 10.0.2.2 "ecmp test $i"; -sport $i; done`" xterm's h1 terminal, we can track through wireshark that packets have been sent through switch2 and switch3 randomly.

Task 3:

The first step was adding counters. In P4 we can use a register array, so I created one with an entry for each possible egress port. Each entry is a 64-bit counter so it won't overflow quickly. In the egress pipeline, every time a packet is about to leave, the program looks up the counter for that port and adds the packet's length. This way, the register always reflects the total bytes that actually left through each port.

For the query mechanism. Instead of involving the control plane, I decided to make a special type of packet. I reserved protocol number 253 in the IPv4 header, and defined a simple "monitor header" as the payload. This header includes an opcode (query or reply), a port index, and a value field. When a host sends one of these packets with "query" set, the switch recognizes it in the parser, jumps into a special path in ingress, reads the correct register entry, writes that value back into the header, changes the opcode to "reply," and then bounces the packet back to the sender.

On the host side, I used a short Python script with Scapy to send the query packet and wait for the reply (hw1-3/monitor_packets.py). When the reply comes back, it contains the current counter for the port that was requested. Testing this by generating traffic first and then sending queries showed that the counts updated as expected.

To test the program, first send the packets with "`for i in $(seq 1 50); do python send.py 10.0.2.2 "$i"; done;`". Then the packet will include the 253 header. After that, track down the packet with "`tcpdump -i eth0 'ip proto 253' -vv -XX & python`

`hw1-3/monitor_packets.py 2 --iface eth0 --dst 10.0.1.1`" with 2 referring to port 2.

Milestone 2:

Task1:

I wrote an ecmp_random_flow.py file inside hw2-1 folder, what it will do is:

1. send proto 253 monitor query to read the 64-bit egress byte counters on the two ecmp ports
2. generate random flows TCP packet to the destination(10.0.2.2). Each packet has a random source IP, random TCP source port, destination port, and a random payload size from 100, 500, 1000, 1400(max is 1500). These fields drive the ECMP 5-tuple hash.
3. Read the counters again, query the same two ports, compute deltas, and print the report (upper bytes, lower bytes, total, and percentages). It also prints an approximate on-wire total measured at the sender.

```
=== ECMP Report ===
Source total bytes (approx on-wire): 85000
Upper path bytes: 40284  (47.4%)
Lower path bytes: 44716  (52.6%)
root@p4dev:/home/p4/tutorials/exercises/basic# python send.py 10.0.2.2 "receive?"
sending on interface eth0 to 10.0.2.2
```

The deltas of the two egress counters represent bytes forwarded via the **upper** and **lower** next hops during the test. With randomized 5-tuples, ECMP split was ~50/50 (minor variation expected from randomness and header size differences). The sender's "approx total" (L2+L3+L4) should be close to the sum of the two path deltas; small differences can arise from L2 overheads, ARP, or other ports.

in mininet h1 terminal: "`python ./hw2-1/ecmp_random_flow.py -port 2 3 -iface eth0`" to run the program

Task2:

From changing flow-based ECMP to packet-based ECMP, the load balancing has improved. I sent 5000 packets with packet-based ECMP to make sure the result is accurate:

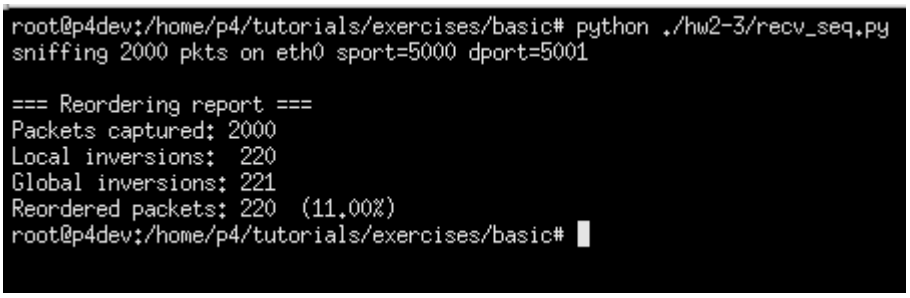
```
=== ECMP Report ===
Source total bytes (approx on-wire): 34279640
Upper path bytes: 17214512  (50.2%)
Lower path bytes: 17065128  (49.8%)
root@p4dev:/home/p4/tutorials/exercises/basic#
```

What I have changed is I added a 1-entry register pkt_seq and replaced the ECMP selection action with a per-packet selector. On every IPv4 packet in ingress, I read pkt_seq, increment it, and map that sequence to an ECMP member using hash(..., {seq}, ecmp_count). The selected member index is ecmp_base + hv, which feeds the existing ecmp_nhop table. This

makes each packet eligible to take a different path (round-robin/randomized), rather than a flow-based hashing approach.

```
python ./hw2-1/ecmp_random_flow.py -port 2 3 -iface eth0 –flows 5000
```

Task 3:



Using a single TCP flow with per-packet load balancing and a sequence tag, I captured 2000 packets at the destination and quantified reordering by inversions. I observed 220 local inversions, 221 global inversions, and 220 reordered packets (11.0%). Thus, per-packet load balancing introduced an ~11% out-of-order rate in this test, with reordering dominated by adjacent swaps.

To run this test:

On h2:

```
python ./hw2-3/recv_seq.py
```

Filter the same 5-tuple (sport=5000, dport=5001), extract the first 8 bytes from each packet, producing the arrival sequence A = [a1, …, ak]. Because this isolates the flow we care about and captures the order in which packets actually arrived.

On h1:

```
python ./hw2-3/send_seq.py
```

I keep a fixed 5-tuple (10.0.1.1:5000 → 10.0.2.2:5001, TCP).

Per-packet LB should split packets even within the same flow; keeping the 5-tuple constant ensures we are measuring intra-flow reordering.

Every packet's payload starts with an 8-byte big-endian sequence number 0,1,2,.... because the data plane doesn't know "original order." Tagging the order at the source lets the receiver reconstruct it regardless of the path.

Sleep 0.05 every 200 packets to prevent the overflow of the buffer, and I have set the latency from switch 1 to switch 2 to 1ms, and switch 1 to switch 3 to 20ms, to signify the difference.

Milestone 3:

Task 1:

what I did is I added two register on S1:

- fl_last_ts: last seen arrival timestamp for a flow bucket
- fl_choice: the ecmp member pinned for the current flowlet

also I defined a bucket index, which is a stable hash of the 5-tuple maps each flow to a bucket (flowlet_buckets = 4096)

Selection action. set_flowlet_select(ecmp_base, ecmp_count, timeout_us):

Read last and pinned for the flow's bucket. Compute gap = now – last from standard_metadata.ingress_global_timestamp. If gap > timeout_us (flowlet gap). Start a new flowlet: pick a member using a counter→hash and store it in fl_choice; update fl_last_ts. Else, reuse pinned. Set meta.ecmp_select = ecmp_base + member; and let ecmp_nhop do the rewrite.

Task 2:

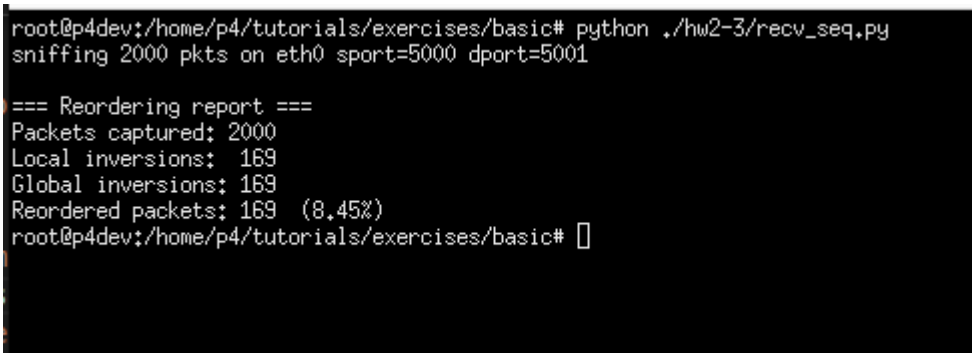
I use recv_seq.py and send_seq.py in hw2-3 folder for this assignment too. Since the scapy code inside already contains the timeout function. just run:
h1:

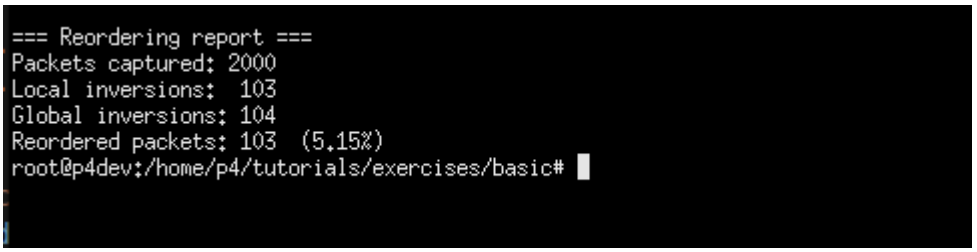
```
python ./hw3-1,3-2/send_seq.py
```

h2:

```
python ./hw3-1,3-2/recv_seq.py
```

Scheme / Timeout	Local inv.	Global inv.	Reordered pkts	Reorder rate
Per-packet ECMP (baseline)	220	221	220	11.0%
Flowlet (10 ms)	169	169	169	8.45%
Flowlet (25 ms)	103	104	103	5.15%





About the files:
I updated all my code in same basic.p4 file. If you want the previous version of the code, it is in my github: https://github.com/Andrew513/Comp536-git/commits/main/.
All of my code is inside the basic folder in the exercise.
I have the folder named as the id of the question: hw1-2: milestone 1 and task2.
And for milestone 3, the update is in basic.p4